Endpoint Detection for Streaming ASR with Phoneme Modeling

Zhiwei Xie
College of Computer Science and Technology
Xinjiang University
Urumqi, Xinjiang, China
zhiwei_powerful@163.com

Chengcheng He
College of Computer Science and Technology
Xinjiang University
Urumqi, Xinjiang, China
chengchengxju@gmail.com

Yanbing Li*
Xinjiang Laboratory of Multi?Language Information Technology
Xinjiang University
Urumqi, Xinjiang, China
liyb@xju.edu.cn

Wushouer Silamu
Xinjiang Laboratory of Multi?Language Information Technology
Xinjiang University
Urumqi, Xinjiang, China
wushour@xju.edu.cn

Abstract

Endpoint detection in speech recognition identifies when a natural sentence stops during recognition. Recent research on integrating endpoint detection and speech recognition into a single streaming ASR model is limited, with past methods often having high detection latency due to coarse modeling granularity. Therefore, we propose using phoneme information to model endpoint detection to assist the speech recognition system, achieving lower latency in endpoint detection for RNN-T-based streaming ASR systems. Validation on end-to-end endpoint detection-oriented speech recognition tasks shows that, compared to baseline methods, the phoneme-branch-based ASR system significantly reduces endpoint detection latency, improves coverage, and maintains speech recognition performance at the cost of only a slight increase in parameters and real-time factor.

CCS Concepts

• **Computing methodologies** → Artificial intelligence; Natural language processing.

Keywords

Speech Recogniton, Endpoint Detection, Phoneme Modeling

ACM Reference Format:
Zhiwei Xie, Chengcheng He, Yanbing Li, and Wushouer Silamu. 2024. Endpoint Detection for Streaming ASR with Phoneme Modeling. In *2024 5th International Conference on Computer Science and Management Technology (ICCSMT 2024), October 18–20, 2024, Xiamen, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3708036.3708091

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCSMT 2024, Xiamen, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0999-9/24/10
https://doi.org/10.1145/3708036.3708091

1 Introduction

Endpoint detection in speech recognition identifies when a natural sentence stops during recognition. It is crucial for applications like voice commands and online subtitles that require real-time sentence end detection. Early methods used some modules like VAD [1] (Voice Activity Detection) and CTC [20] (Connectionist Temporal Classification) to detect silence frames as endpoints [2-5]. However, detecting silence frames is inherently different from endpoint detection. Later methods introduced independent endpoint detection models [6-9], but these had drawbacks like high latency and model redundancy. Recently, research has focused on integrating endpoint detection with speech recognition [10-14], aiming for a unified system that handles both tasks simultaneously.

For the end-to-end streaming speech recognition model oriented towards endpoint detection, two methods play an important role in this paper. Shuo-Yiin Chang et al·[10] introduced a special token like </s> in the output text token set of speech recognition to mark endpoints, this approach can easily affect the performance of the speech recognition itself. W Ronny Huang et al. [14] proposed using an additional output branch to handle the text token set containing the </s> token while keeping the original output branch unchanged, this method only slightly impacts the performance of speech recognition, but the granularity of the modeling units is coarse, leading to high detection latency. We proposed to address the issue of coarse modeling unit granularity leading to high detection latency. The contributions of the proposed method are: 1) proposing fine-grained phoneme modeling units for endpoint detection in streaming speech recognition; 2) significantly reducing detection latency with minimal parameter increase and only a slight real-time factor cost. 3) Propose a simple dual-branch decoding process for deployment in production environments.

2 Proposed Method

The proposed method is based on the backbone model shown in Figure 1 (left). Its core idea, proposed by W. Ronny Huang et al. [14] , extends the original RNN-T [19] network by adding an extra joiner network to output token sequences with endpoint detection tokens. For example, if the text corresponding to an audio is "hello, how can I help you?", the text joiner network outputs "HELLO

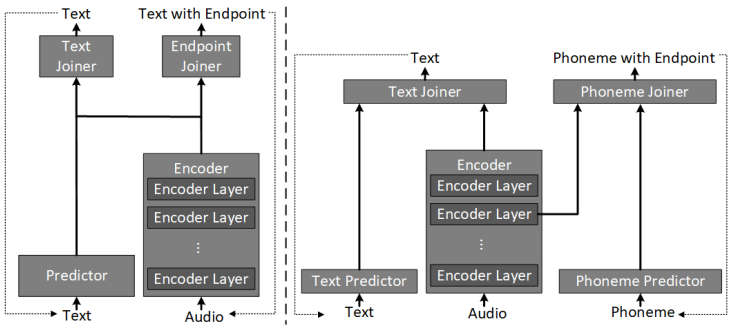

Figure 1: Backbone (left) and proposed model (right).

HOW CAN I HELP YOU", while the endpoint joiner network would output "HELLO </s> HOW CAN I HELP YOU </fs>".

As shown in Figure 1 (right), the proposed model not only includes the phoneme joiner network but also adds a phoneme predictor network. The phoneme predictor network takes the output of the 7th layer of the encoder network as input. These two additional network modules play a role in predicting phoneme sequences with endpoint detection markers. If the text joiner network output is "HELLO HOW CAN I HELP YOU", the phoneme joiner network would output "HH AH0 L OW1 </s> HH AW1 K AE1 N AY1 HH EH1 L P Y UW1 </fs>"(for PhoneEP ASR-EP, later will introduce). The loss function is the sum of the ASR loss functions from both branches.

2.1 Data label processing

The method proposed uses punctuation marks in the text to identify endpoints, which are categorized into two types: mid endpoints (labeled as STOP, corresponding to the output token </s>) and end endpoints (labeled as FSTOP, corresponding to the output token </fs>). The proposed method requires the phoneme transcripts to train the phoneme branch network. Generally, only the text sequence is available, so the corresponding phoneme sequence can be obtained using the G2P [18] tool[1].The entire data labeling process is as follows: 1) Replace the punctuation marks in the text transcripts with the corresponding </s> or </fs>(</fs> for question or period mark, </s> for others), 2) use the G2P tool to generate phonemes for all characters except </s> and </fs>, resulting in a phoneme sequence with endpoint markers. Note that </fs> and </s> are used to distinguish between the two types of punctuation marks for the network, both representing endpoint detection markers.

2.2 Phoneme alignment

In addition to phoneme sequence generation, another required label information is the timing of the corresponding phoneme in the audio sequence. Although this label information does not participate in training, it can provide a reference for the latency of the predicted results, allowing the calculation of the delay between the predicted endpoint and the actual occurrence of the endpoint. This method uses the Montreal Forced Aligner (MFA)[2] to generate the timing of the corresponding phonemes in the audio. After obtaining the phoneme sequence, the end time of the phoneme immediately before </s> or </fs> is taken as the occurrence time of the endpoint.

2.3 Joint decoding for endpoint detection and speech recognition

The proposed method is based on streaming speech recognition. However, the method proposed increases the complexity of the inference process by adding an additional output branch. To address this, a simple and effective algorithm is proposed to handle the increased computational process due to the new branch during streaming inference, ensuring the real-time factor computation of the model in a streaming deployment. The following algorithm describes the decoding process in a decoding step:

Algorithm 1 Joint decoding step for endpoint detection and speech recognition.

for step in decode_steps:

Update text hypothesis **text_hyp** using beam search decoding for text tokens.

Update phoneme hypothesis **phone_hyp** using beam search decoding for phoneme tokens.

If "</fs>" in **phone_hyp**:

Extract phoneme from **text_hyp** get **phone_hyp_from_text_hyp**, using MFA tool.

Align phoneme token sequences between **phone_hyp_from_text_hyp** and **phone_hyp** to get alignment result **ali**.

Based on **ali**, split **text_hyp** and **phone_hyp** into **text_hyp1**, **text_hyp2**, **phone_hyp1**, **phone_hyp2**.

Update beam decoding state with **text_hyp1**, **text_hyp2**, **phone_hyp1**, **phone_hyp2**.

text_hyp1 and **phone_hyp1** as the decoded text and phoneme token set in this step.

3 Experimental Setup

3.1 Datasets

The dataset used for the experiments is LibriTTS [15] which is similar to the LibriSpeech [17] dataset but retaining the punctuation marks in the corresponding book text content and has been

[1]https://github.com/Kyubyong/g2p
[2]https://montrealcorpustools.github.io/

Table 1: Parameter and WER.

Model	Test Set	Parameters	Branch WER	WER
Text ASR	test-clean	75 M	-	6.62
	test-other		-	16.11
TEXT ASR-EP	test-clean	75 M	-	15.21
	test-other		-	31.14
Text2Branch ASR-EP	test-clean	76.24 M	6.69	6.64
	test-other		15.88	15.82
Text2BranchPre ASR-EP	test-clean	76.24 M	6.83	6.62
	test-other		16.59	16.11
PhoneBPE-500 ASR-EP	test-clean	76.48 M	9.11	6.33
	test-other		21.09	**14.87**
PhoneBPE-1500 ASR-EP	test-clean	78.44 M	8.04	**6.14**
	test-other		19.22	14.92
PhoneEP ASR-EP	test-clean	75.65 M	**3.72**	6.47
	test-other		**10.53**	15.26

filtered to remove severely noisy audio. The training subsets train-clean-360 and train-clean-10 were used, with a total duration of approximately 242 hours. Although LibriTTS was originally intended for speech synthesis tasks, it is suitable for training joint speech recognition and endpoint detection models due to its punctuated text transcripts.

In addition to the aforementioned text label processing, an additional processing step was performed on the dataset. This was necessary because proposed model trained on the original LibriTTS data would often stop recognizing subsequent voiced audio upon predicting the </fs> label during actual predictions. The additional processing steps were as follows: first, the dataset was split into two halves, one consisting of long audio segments exceeding 15 seconds and the other of short audio segments less than 15 seconds. The long audio segments remained unchanged, while the short audio segments were concatenated pairwise along with their corresponding text parts. Secondly, after concatenating the short audio segments, a specified number of hours of audio data were randomly selected for further random concatenation, we add additional 100 hours random concatenation for training data, and additional 1 hour for dev-clean, dev-other, test-clean and test-other.

3.2 Model specifications

The models in the experiments are derived from the pruned_transducer_stateless2 model[3] of K2 toolkit [16]. This model is primarily used for non-streaming models, but with simple modifications to a few parameters as provided in the K2 documentation, it can be adapted for training and inference in a streaming manner. For added branch network module, such as additional phoneme predictor and phoneme joiner network, we copy the predictor and joiner network from pruned_transducer_stateless2 itself.

We trained several models to validate the performance: The **Text ASR** is the original streaming pruned_transducer_stateless2 model. **Text ASR-EP** is added additional endpoint tokens, remain model structure same as Text ASR. **Text2Branch ASR-EP** uses the structure of the backbone model, Text2Branch ASR-EP is retrained all at once. "Pre" in **Text2BranchPre ASR-EP** stands for "Pretrained," meaning the model is trained in two stages: first, the Text ASR model is Pretrained, then its parameters are fixed while adding an extra joiner network branch, which is trained on the dataset to get the final Text2BranchPre ASR-EP model, it uses the same training strategy and model structure as backbone model. **PhoneBPE-500 ASR-EP** is the proposed model, using BPE (Byte Pair Encoding) trained from phoneme transcripts as model units, with the number of BPE units being 500. **PhoneBPE-1500 ASR-EP** is the same as PhoneBPE-500 ASR-EP, but with 1500 BPE units. **PhoneEP ASR-EP** uses phonemes themselves as modeling units, with the number of modeling units being 74. The other parts are the same as PhoneBPE-500 ASR-EP and PhoneBPE-1500 ASR-EP.

4 Experimental Results and Analysis

4.1 Analysis of parameter and WER

First, we evaluated the performance of different models in terms of parameter and WER, as shown in Table 1. The first four rows correspond to baseline models. For parameter, the largest one is PhoneBPE-1500 ASR-EP, which only increases by 3.48M compared to the original Text ASR model, indicating minimal impact on parameter increment. Text ASR-EP has no additional tokens or output branches, so it has the same parameter count as the original Text ASR. Text2Branch ASR-EP adds an extra joint network module. PhoneBPE-500 ASR-EP, PhoneBPE-1500 ASR-EP, and Phone ASR-EP add parameters due to additional phoneme prediction and joint networks, differing only in the number of final output units, with 500, 1500, and 74 output units respectively, so PhoneBPE-1500 ASR-EP has the most parameters, and Phone ASR-EP has the least.

Secondly, for WER, The Branch WER represents the WER of the additional output branch after removing </s> and </fs>. For the first four baseline models, Branch WER is the WER of natural language text after removing </s> and </fs>. For the three models in the lower half using phoneme modeling for the additional output branch, Branch WER is the WER of phoneme text after removing </s> and </fs>. Similarly, WER represents the WER of text from

[3]https://github.com/k2-fsa/icefall/tree/master/egs/librispeech/ASR/pruned_transducer_stateless2

Table 2: Coverage of STOP.

Model	Test Set	STOP Coverage	FSTOP Coverage
Text ASR	test-clean	-	-
	test-other	-	-
TEXT ASR-EP	test-clean	56.73	41.92
	test-other	58.3	38.39
Text2Branch ASR-EP	test-clean	**76.91**	40.46
	test-other	**78.05**	38.5
Text2BranchPre ASR-EP	test-clean	69.87	52.65
	test-other	72.64	49.04
PhoneBPE-500 ASR-EP	test-clean	74.58	67.7
	test-other	75.06	62.64
PhoneBPE-1500 ASR-EP	test-clean	74.27	45.68
	test-other	74.77	43.04
PhoneEP ASR-EP	test-clean	72.74	**82.36**
	test-other	71.67	**77.09**

Table 3: Latency of ASR units and STOP units.

Model	Test Set	Branch TextMean Latency	Text Mean Latency	STOP Mean Latency	FSTOP Mean Latency
Text ASR	test-clean	-	318	-	-
	test-other	-	314	-	-
TEXT ASR-EP	test-clean	-	**204**	207	**58**
	test-other	-	**199**	221	**62**
Text2Branch ASR-EP	test-clean	312	320	289	167
	test-other	308	316	302	172
Text2BranchPre ASR-EP	test-clean	310	318	322	184
	test-other	307	314	338	186
PhoneBPE-500 ASR-EP	test-clean	183	343	149	86
	test-other	179	339	156	90
PhoneBPE-1500 ASR-EP	test-clean	218	345	177	105
	test-other	214	341	186	109
PhoneEP ASR-EP	test-clean	**144**	351	**133**	130
	test-other	**146**	347	**135**	135

the original output branch after removing </s> and </fs>, the same as conventional WER. Table 1 shows that TEXT ASR-EP performs significantly worse than other models due to directly incorporating special tokens </s> and </fs> into a single output branch. For WER, PhoneBPE-1500 ASR-EP and PhoneBPE-500 ASR-EP achieve the best results on test-clean and test-other, with other models based on additional phoneme output branches not far behind. Models with additional phoneme output branches generally have better WER than the upper models, indicating that adding an additional phoneme output branch does not impair and even slightly improves speech recognition performance. For Branch WER, PhoneEP ASR-EP significantly outperforms other models, analysis on predictions reveals that other models tend to stop predicting subsequent text at </fs> in the additional output branch despite data concatenation. Overall, PhoneEP ASR-EP outperforms other methods in WER analysis.

4.2 Analysis of coverage

Table 2 shows the coverage of middle endpoints (STOP) and end endpoints (FSTOP). Coverage rate is defined as the number of detected endpoints divided by the total number of endpoints. The results indicate that for end endpoint coverage, the PhoneEP ASR-EP model significantly outperforms other models. For middle endpoint coverage, the Text2Branch ASR-EP model has the best rate, but other models with additional phoneme output branches are not far behind. Since endpoint detection prioritizes end endpoint coverage, the PhoneEP ASR-EP model is superior in this metric.

4.3 Analysis of latency

The latency analysis of each model is shown in Table 3. Mean latency is calculated as the time from when an endpoint is detected to when it actually occurs, divided by the total number of detected endpoints. The actual endpoint times are aligned with the MFA text and audio results. The experiment shows that TEXT ASR-EP

Table 4: Percent line and common latency.

Model	Test Set	STOPP90	STOPP99	FSTOPP90	FSTOP P99	STOP Common Latency(613|366)	FSTOPCommon Latency(9|4)
TEXT ASR-EP	test-clean	390	690	**120**	**200**	250	88
	test-other	420	860	**120**	**210**	251	**113**
Text2Branch ASR-EP	test-clean	480	730	250	330	365	204
	test-other	519	869	240	320	350	253
Text2BranchPre ASR-EP	test-clean	530	829	250	339	390	240
	test-other	589	1030	250	339	375	243
PhoneBPE-500 ASR-EP	test-clean	250	400	160	240	186	124
	test-other	270	510	160	240	172	173
PhoneBPE-1500 ASR-EP	test-clean	299	439	189	259	224	160
	test-other	319	549	179	250	216	183
PhoneEP ASR-EP	test-clean	**229**	**369**	280	359	**159**	**76**
	test-other	**240**	**429**	270	359	**103**	173

has the lowest text mean latency, while models with additional phoneme output branches generally have an average latency about 125 milliseconds higher. This suggests that adding phoneme output branches introduces additional latency for original ASR branch, but note that TEXT ASR-EP model has the worst WER performance. Branch Text Mean Latency for models with additional phoneme output branches is generally lower, indicating that the finer granularity of phoneme branches can reduce token prediction latency and the detection latency of </s> and </fs>. Subsequent experiments will support this hypothesis.

STOP Mean Latency and FSTOP Mean Latency are endpoint detection latencies, crucial metrics for endpoint detection. The calculation method is the same as for text mean latency, but the target tokens differ. Results show that except for the TEXT ASR-EP model, models with additional phoneme output branches generally have lower endpoint detection latencies than those without. For middle endpoint latency, PhoneEP ASR-EP has the lowest latency. For FSTOP mean latency, excluding TEXT ASR-EP, the PhoneBPE-500 ASR-EP model has the lowest latency. Although TEXT ASR-EP has low latency, it comes at the cost of speech recognition performance.

It's worth noting that the mean endpoint latency here is the number of detected endpoints divided by the total number of endpoints. Since different models detect different numbers of endpoints, comparing endpoint detection latency this way may be unfair. Therefore, it's necessary to compare the mean latency of the commonly detected endpoints.

4.4 Analysis of percent line and common latency

The last two columns in Table 4 show the mean latency of commonly detected endpoints for models with endpoint detection. The numbers below the headers indicate the number of commonly detected endpoints, e.g., (613|366) means 613 and 366 middle endpoints were commonly detected in the test-clean and test-other datasets, respectively. The number of common end endpoints is low because the endpoint coverage rates of models other than PhoneEP ASR-EP are very low. From the table, we see that the TEXT ASR-EP model has the lowest end endpoint latency but at the cost of speech recognition accuracy, making it impractical. Excluding TEXT ASR-EP, PhoneEP ASR-EP has the lowest latency.

The middle four columns of Table 4 show the percentile metrics of endpoint detection. With P90 as an example, the P90 line is a value below which 90% of the data points fall. For example, if P90 is 100, it means 90% of data points are less than or equal to 100, while 10% are greater. The results show that, excluding TEXT ASR-EP, PhoneBPE-500 ASR-EP has the best end endpoint percentile metrics, and PhoneEP ASR-EP has the best middle endpoint percentile metrics. For end endpoints, the higher percentile metrics for PhoneEP ASR-EP may be due to its higher coverage rate, resulting in a larger sample size compared to other models. This is also reflected in the middle endpoint percentiles where coverage rates are similar across models, making the middle endpoint percentile data more representative.

Combining the results from Tables 3 and 4, PhoneEP ASR-EP is the best model in terms of latency metrics.

4.5 Analysis of RTF

To assess the impact of adding extra output branches on real-time performance, we used the Sherpa tool from K2 to deploy the models and measured the real-time factor (RTF) on an RTX 3090 GPU machine, as shown in Table 5. "PhoneEP ASR-EP No Endpointer" indicates inference using the PhoneEP ASR-EP model with endpoint detection turned off. Several conclusions can be drawn from the table:1) Comparing PhoneEP ASR-EP with PhoneEP ASR-EP No Endpointer, adding endpoint detection reduces the RTF. This may be because endpoint detection keeps the decoding sequence length in memory controlled, reducing decoding overhead. 2) The Text ASR, using the rule-based long audio segmentation decoding strategy mentioned before. Compare Text ASR-EP to Text ASR, result shows that more accurate endpoint detection can reduce the RTF, likely due to shorter decoding sequences in memory, resulting in lower overhead. 3) Comparing Text ASR, TEXT ASR-EP,

Table 5: Percent line and common latency.

Text ASR	TEXT ASR-EP	Text2Branch ASR-EP
0.1805	0.1776	0.2016
Text2BranchPre ASR-EP	PhoneBPE-500 ASR-EP	PhoneBPE-1500 ASR-EP
0.2055	0.213	0.2137
PhoneEP ASR-EP	PhoneEP ASR-EP No Endpointer	
0.2081	0.2745	

Text2Branch ASR-EP, PhoneBPE-500 ASR-EP, PhoneBPE-1500 ASR-EP, and PhoneEP ASR-EP, adding extra output branches slightly increases the RTF but remains within an acceptable range. 4) Comparing PhoneBPE-500 ASR-EP, PhoneEP ASR-EP, and PhoneBPE-1500 ASR-EP, there is no significant change in the RTF, indicating that the number of output tokens does not strongly affect the RTF.

5 Conclusion

We propose a dual-branch output RNN-T model structure that uses phonemes as an additional output of speech recognition. By leveraging fine-grained phoneme information, we model the output units of the endpoint detection branch and provide a decoding process that outputs endpoint detection tokens along with the text sequence. Experiments show that this phoneme-based dual-branch output network structure significantly reduces endpoint detection latency in end-to-end speech recognition. It achieves this at the cost of a minimal increase in parameters and RTF, without affecting and even improving speech recognition performance.

Acknowledgments

This work was supported by the Research on the Industrial Application of "Complex Network Behavior Analysis, Prediction, and Intervention in a Multilingual Big Data Environment". Project under Grants GJGJZD20210408092806017 and the Research on Explainable Reasoning Methods for Cross-Media Intelligent Question Answering in Online Education. Project under Grants 62137002.

References

[1] Hughes, Thad, and Keir Mierle. "Recurrent neural networks for voice activity detection." 2013 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, 2013.

[2] Hariharan, Ramalingam, Jula Hakkinen, and Kari Laurila. "Robust end-of-utterance detection for real-time speech recognition applications." 2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No. 01CH37221). Vol. 1. IEEE, 2001.

[3] Thomas, Samuel, *et al.* "Improvements to the IBM speech activity detection system for the DARPA RATS program." 2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2015.

[4] Eyben, Florian, *et al.* "Real-life voice activity detection with lstm recurrent neural networks and an application to hollywood movies." 2013 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, 2013.

[5] Yoshimura, Takenori, *et al.* "End-to-end automatic speech recognition integrated with CTC-based voice activity detection." ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2020.

[6] Shannon, Matt, *et al.* "Improved End-of-Query Detection for Streaming Speech Recognition." Interspeech. 2017.

[7] Chang, Shuo-Yiin, *et al.* "Endpoint Detection Using Grid Long Short-Term Memory Networks for Streaming Speech Recognition." Interspeech. 2017.

[8] Maas, Roland, *et al.* "Combining acoustic embeddings and decoding features for end-of-utterance detection in real-time far-field speech recognition systems." 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2018.

[9] Hwang, Inyoung, and Joon-Hyuk Chang. "End-to-end speech endpoint detection utilizing acoustic and language modeling knowledge for online low-latency speech recognition." IEEE access 8, 2020, 161109-161123.

[10] Chang, Shuo-Yiin, *et al.* "Joint endpointing and decoding with end-to-end models." ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2019.

[11] Li, Bo, *et al.* "Towards fast and accurate streaming end-to-end ASR." ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2020.

[12] Lu, Liang, Jinyu Li, and Yifan Gong. "Endpoint detection for streaming end-to-end multi-talker ASR." ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2022.

[13] Chang, Shuo-Yiin, *et al.* "Joint endpointing and decoding with end-to-end models." ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2019.

[14] W. Ronny Huang, Shuo-Yiin Chang, David Rybach, Tara N. Sainath, Rohit Prabhavalkar, Cal Peyser, Zhiyun Lu, Cyril Allauzen: E2E Segmenter: Joint Segmenting and Decoding for Long-Form ASR. INTERSPEECH 2022, 4995-4999.

[15] Heiga Zen, Viet Dang, Rob Clark, Yu Zhang, Ron J. Weiss, Ye Jia, Zhifeng Chen, Yonghui Wu: LibriTTS: A Corpus Derived from LibriSpeech for Text-to-Speech. INTERSPEECH, 2019, 1526-1530.

[16] Fangjun Kuang, Liyong Guo, Wei Kang, Long Lin, Mingshuang Luo, Zengwei Yao, Daniel Povey: Pruned RNN-T for fast, memory-efficient ASR training. INTERSPEECH 2022, 2068-2072.

[17] Panayotov, Vassil, *et al.* "Librispeech: an asr corpus based on public domain audio books." 2015 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, 2015.

[18] Pine, Aidan, *et al.* "G2P: Rule-based, index-preserving grapheme-to-phoneme transformations." Proceedings of the Fifth Workshop on the Use of Computational Methods in the Study of Endangered Languages. 2022.

[19] Graves, Alex. "Sequence transduction with recurrent neural networks." arXiv preprint arXiv:1211.3711, 2012.

[20] Graves, Alex, *et al.* "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks." Proceedings of the 23rd international conference on Machine learning. 2006.